Constructor / Destructor Order in Inheritance

The golden rule is:

> **Construction: Base → Derived**

> **Destruction: Derived → Base**

Think:

```
Constructor:
Base class
   ↓
Derived class

Destructor:
Derived class
   ↓
Base class
```

---

1. Simple example

```cpp
class Animal {
public:
    Animal() {
        cout << "Animal constructor\n";
    }

    ~Animal() {
        cout << "Animal destructor\n";
    }
};

class Dog : public Animal {
public:
    Dog() {
        cout << "Dog constructor\n";
    }

    ~Dog() {
        cout << "Dog destructor\n";
    }
};
```

Now:

```cpp
Dog d;
```

Output:

```
Animal constructor
Dog constructor

Dog destructor
Animal destructor
```

Why?

A `Dog` **is an Animal**, so the `Animal` part must be constructed before the `Dog`-specific part.

When destroying, the reverse happens:

```
Destroy Dog-specific part
        ↓
Destroy Animal part
```

This guarantees that the derived class can safely use its base-class portion while being destroyed.

---

2. What if there are multiple levels?

```cpp
class A {
public:
    A() { cout << "A\n"; }
    ~A() { cout << "~A\n"; }
};

class B : public A {
public:
    B() { cout << "B\n"; }
    ~B() { cout << "~B\n"; }
};

class C : public B {
public:
    C() { cout << "C\n"; }
    ~C() { cout << "~C\n"; }
};
C obj;
```

Output:

```
A
B
C

~C
~B
~A
```

So:

```
Construction:
A → B → C

Destruction:
C → B → A
```

---

3. What about data members?

This is where interviewers often test you.

Consider:

```cpp
class Base {
    Member m1;

public:
    Base() {
        cout << "Base\n";
    }
};

class Derived : public Base {
    Member m2;

public:
    Derived() {
        cout << "Derived\n";
    }
};
```

Construction order is:

```
1. Base class
2. Derived's members
3. Derived constructor body
```

More precisely:

```
Base's base classes
↓
Base's data members
↓
Base constructor body
↓
Derived's base classes
↓
Derived's data members
↓
Derived constructor body
```

For destruction, everything is reversed.

---

4. Actual complete order

Suppose:

```
class A {
    Member a;
};

class B : public A {
    Member b;
};
```

Creating:

```
B obj;
```

Order:

```
A's base classes
A's members
A constructor body

B's members
B constructor body
```

Destroying:

```
B destructor body
B's members
A destructor body
A's members
```

The key principle is:

> **Construction follows declaration/inheritance order. Destruction happens in exact reverse order.**

---

5. Multiple inheritance ⭐

This is another common interview question.

```
class A {
public:
    A() { cout << "A\n"; }
};

class B {
public:
    B() { cout << "B\n"; }
};

class C : public A, public B {
public:
    C() { cout << "C\n"; }
};
C obj;
```

Output:

```
A
B
C
```

Destruction:

```
~C
~B
~A
```

Important:

The order is determined by the **inheritance declaration order**, NOT the order in the constructor initializer list.

```
class C : public A, public B
```

means:

```
A → B → C
```

---

6. Constructor initializer list trap

Consider:

```
class C : public A, public B {
public:
    C() : B(), A() {
        cout << "C\n";
    }
};
```

Some beginners expect:

```
B
A
C
```

But that's **wrong**.

Actual:

```
A
B
C
```

Because base classes are initialized according to their **declaration order**:

```
class C : public A, public B
```

not initializer-list order.

Interview answer

> Base classes are initialized in the order in which they appear in the class declaration, regardless of the order in the constructor initializer list.

---

7. Virtual destructor

This is extremely important.

Consider:

```
class Base {
public:
    ~Base() {
        cout << "Base destructor\n";
    }
};

class Derived : public Base {
public:
    ~Derived() {
        cout << "Derived destructor\n";
    }
};
```

Now:

```
Base* ptr = new Derived();
delete ptr;
```

If `Base`'s destructor is **not virtual**, deleting through a base pointer results in **undefined behavior**.

For polymorphic base classes, use:

```
class Base {
public:
    virtual ~Base() {
        cout << "Base destructor\n";
    }
};
```

Now:

```
Base* ptr = new Derived();
delete ptr;
```

correctly produces:

```
Derived destructor
Base destructor
```

Why?

Because `delete ptr` needs to know the **actual runtime type** so it can destroy the derived portion first.

---

8. The classic interview question

Q: Why should a base class destructor be virtual?

Answer:

> If an object of a derived class is deleted through a base-class pointer, the base destructor must be virtual to ensure the derived destructor is called first, followed by the base destructor. Otherwise, deleting through the base pointer causes undefined behavior.

Example:

```
Base* p = new Derived();
delete p;
```

For a polymorphic base:

```
virtual ~Base() = default;
```

---

9. Constructor calls with virtual functions — another trap

Interviewers sometimes combine these topics.

```
class Base {
public:
    Base() {
        foo();
    }

    virtual void foo() {
        cout << "Base\n";
    }
};

class Derived : public Base {
public:
    void foo() override {
        cout << "Derived\n";
    }
};
Derived d;
```

The constructor calls:

```
Base
```

not:

```
Derived
```

Why?

During base construction, the derived portion **hasn't been constructed yet**.

Similarly, during destruction, virtual dispatch does not behave as if the derived object were still fully alive.

Interview rule:

> **Don't rely on virtual dispatch inside constructors/destructors.**

Must-remember interview cheat sheet

```
CONSTRUCTION
____________
Base classes
    ↓
Data members
    ↓
Constructor body


DESTRUCTION
___________
Destructor body
    ↓
Data members
    ↓
Base classes
```

For inheritance:

```
Constructor:  Base → Derived
Destructor:   Derived → Base
```

For multiple inheritance:

```
class C : public A, public B
```

gives:

```
A → B → C
C → B → A
```

For polymorphic deletion:

```
class Base {
public:
    virtual ~Base() = default;
};
```

The 4 questions I'd expect in an interview

1. **What is the order of constructor/destructor calls in inheritance?**
2. **What happens with multiple inheritance?**
3. **Does initializer-list order determine base construction order? → No, declaration order does.**
4. **Why should a base destructor be virtual? → Safe deletion through a base pointer.**